

Comparative Analysis of HTTP/1.1, HTTP/2, and HTTP/3

1st Diogo Valverde
dept. of Computer Science
FCUP
Porto, Portugal
up202509119@up.pt

2nd Luís Freitas
dept. of Computer Science
FCUP
Porto, Portugal
up201905767@up.pt

3rd Pedro Mariano
dept. of Computer Science
FCUP
Porto, Portugal
up202509341@up.pt

Abstract—This report presents the initial stage of a project focused on the analysis of HTTP/1.1, HTTP/2, and HTTP/3. The main objective is to compare the performance of these protocol versions under different network conditions through quantitative measurements. Particular attention is given to HTTP/3 and its connection migration capability in scenarios involving a change of access network, such as from WiFi to 5G. This report summarizes the technological background, defines the project objectives, identifies the tools to be used, presents preliminary evidence of feasibility, and outlines the plan for the remaining work.

Index Terms—HTTP/1.1, HTTP/2, HTTP/3, QUIC, performance evaluation, connection migration, WiFi, 5G, network measurement

I. INTRODUCTION

The Hypertext Transfer Protocol (HTTP) has evolved to meet the requirements of modern Web applications, which demand low latency, efficient delivery of multiple resources, and robustness under heterogeneous network conditions. In this evolution, HTTP/1.1, HTTP/2, and HTTP/3 represent three major stages of Web communication [1]–[3].

HTTP/1.1 remained dominant for many years, but its performance is limited in scenarios with many embedded resources. HTTP/2 improved efficiency through binary framing, multiplexing, and header compression [2], [4]. However, it still relies on TCP, so packet loss at the transport layer may affect multiple concurrent streams. HTTP/3 addresses this issue by running over QUIC, which was designed to reduce connection establishment delay and improve performance under more challenging conditions [3], [5]–[7].

A particularly relevant feature of QUIC is connection migration, which allows a communication session to continue even when the underlying access network changes. This is especially important in mobile scenarios, such as transitions from WiFi to 5G [5], [8].

The objective of this project is to compare HTTP/1.1, HTTP/2, and HTTP/3 through quantitative measurements under different network conditions, with particular emphasis on evaluating the usefulness of HTTP/3 connection migration.

II. DESCRIPTION OF THE TECHNOLOGY

A. HTTP/1.1

HTTP/1.1 improved on earlier HTTP behavior by introducing persistent connections, which allow multiple objects

to be transferred through a single TCP connection without repeatedly opening and closing it. It also introduced request pipelining, allowing a client to send multiple requests without waiting for each individual response. Performance studies confirmed that both mechanisms reduced connection overhead and network traffic [1], [9].

Despite these improvements, HTTP/1.1 still relies on a fundamentally sequential communication model and does not provide true multiplexing within a single connection. In practice, responses must still be delivered in order, which limits the efficient parallel delivery of multiple resources. As Web pages became more complex and started to include larger numbers of embedded objects, such as images, scripts, and style sheets, these limitations became more evident. Although HTTP/1.1 improved efficiency relative to previous versions, its communication model remained insufficient for modern Web workloads and later motivated the design of HTTP/2.

A fundamental limitation of this model is head-of-line blocking. When multiple requests share the same HTTP/1.1 connection, responses are constrained by in-order delivery at both the application and transport layers. A delayed response or a lost TCP segment can stall the delivery of all subsequent responses on that connection, even when their data is otherwise available, because TCP guarantees reliable and ordered delivery before passing data to the application [1]. In practice, clients mitigated this problem by opening multiple parallel TCP connections to the same server. This reduced blocking to some extent, but it also introduced additional connection overhead and increased resource consumption at both endpoints.

Because request pipelining only partially mitigated the concurrency limitations of HTTP/1.1, browsers historically relied on multiple parallel TCP connections to the same server in order to fetch different resources concurrently. Although this workaround improved page-loading behavior in practice, it also increased competition between flows, raised connection-management overhead, and made the overall communication pattern less efficient from the network point of view. In this sense, part of the apparent practicality of HTTP/1.1 in real Web deployments depended not on the protocol itself, but on compensating strategies implemented by clients [2].

B. HTTP/2

HTTP/2 was introduced to overcome some of the main inefficiencies of HTTP/1.1, particularly in the context of modern Web pages composed of many embedded resources. Rather than changing the semantics of HTTP, HTTP/2 redefines how messages are framed and exchanged, with the objective of improving resource delivery efficiency and reducing protocol overhead. Its design introduced a binary framing layer, stream multiplexing within a single connection, and header compression [2], [4].

A central difference from HTTP/1.1 is that HTTP/2 allows multiple streams to coexist within the same TCP connection, thereby reducing the need for multiple parallel connections at the application level [2], [4]. This design improves the utilization of the underlying transport connection and enables a more efficient delivery of Web content, especially when a page requires the transfer of many small objects.

Header compression in HTTP/2 is achieved through HPACK, which uses a predefined static table together with a dynamic table maintained during the connection to represent repeated header fields more efficiently. Rather than retransmitting the same headers on every request, HPACK reuses indexed entries from these tables, significantly reducing overhead in workloads where requests share common header fields such as cookies, authentication tokens, or user agent strings [4], [14].

HTTP/2 also introduced stream prioritization, allowing clients to indicate that some resources should be delivered before others within the same connection. In principle, this helps more critical objects, such as stylesheets or scripts needed for initial page rendering, to be transferred earlier. However, the practical benefits of this mechanism depend on how prioritization is implemented at the endpoints, and they are not always fully realized in real deployments [2], [10].

It is important to distinguish the two different forms of head-of-line blocking involved in the transition from HTTP/1.1 to HTTP/2. In HTTP/1.1, the problem exists at the application layer, because responses are effectively constrained by the sequential use of the connection. HTTP/2 removes this application-layer limitation by allowing multiple logical streams to progress concurrently within the same connection. However, all these streams still share a single TCP connection, so the protocol remains subject to transport-layer blocking imposed by TCP itself. Because TCP guarantees reliable and ordered delivery of its byte stream, a single lost packet causes subsequent data to be withheld from the application until the missing segment is retransmitted and reordered, regardless of which HTTP stream that data belongs to. As a result, HTTP/2 solves one form of blocking but remains exposed to another, which limits the practical gains of multiplexing under lossy network conditions [2], [4], [10]. This remaining limitation directly motivated the design of HTTP/3.

C. HTTP/3

HTTP/3 represents a more significant architectural change than the transition from HTTP/1.1 to HTTP/2. Rather than

changing the semantics of HTTP, it maps HTTP onto QUIC, a transport protocol that operates over UDP instead of TCP. This preserves application-layer features such as multiplexed resource delivery while moving transport functionality into QUIC itself, resulting in a more integrated design that combines stream multiplexing, security through TLS 1.3, and connection establishment mechanisms [3], [5]–[7], [11].

A key consequence of this architecture is the elimination of the connection-wide head-of-line blocking that remained in HTTP/2. Unlike TCP, which exposes a single ordered byte stream for the entire connection, QUIC manages multiple independent streams at the transport layer. If a packet belonging to one stream is lost, only that stream is delayed, while the others can continue to deliver data to the application. This directly addresses the limitation of HTTP/2, where a single lost TCP segment could stall all concurrent streams sharing the same connection [3], [5]–[7].

The connection establishment process is also significantly optimized. Traditional TCP-based HTTPS connections require a TCP handshake followed by a separate TLS negotiation, resulting in multiple round-trips before application data can be fully exchanged. QUIC integrates transport and cryptographic handshakes using TLS 1.3, allowing connection establishment in one round-trip for a new connection and potentially zero round-trips for subsequent connections to the same server when previous session information is available [6], [7], [11]. This design may reduce startup delay and is particularly relevant in environments where latency is non-negligible.

Another important aspect of QUIC is that transport functionality is implemented in user space rather than being tied to the operating system kernel, as is traditionally the case with TCP. This design choice allows protocol improvements and implementation updates to be deployed more rapidly, without requiring changes to kernel-level transport stacks. For an evolving protocol such as HTTP/3, this is relevant because performance optimization, congestion-control refinement, and implementation maturity can progress more quickly than in the traditional TCP-based model [5]–[7].

A particularly relevant feature of HTTP/3 for the present project is connection migration. TCP identifies connections through the network four-tuple, so a change of client IP address, such as a transition from WiFi to 5G, breaks the connection and requires reestablishment. QUIC instead uses Connection Identifiers (CIDs), which are independent of the underlying network path. When the client changes its address, these identifiers allow the existing connection to be maintained over the new path without requiring a full connection reestablishment [5], [8]. For mobile communication scenarios, this feature is especially important because it may reduce disruption during access-network changes and preserve application continuity.

This capability is also relevant in situations that do not correspond to a full physical movement between access networks. For example, changes in the observed client address may result from NAT rebinding or other path changes in the underlying network. Because QUIC decouples connection identity from

the client IP address and port combination, it can remain more robust under these events than TCP-based communication. This makes HTTP/3 particularly attractive for mobile and heterogeneous environments, where path continuity cannot always be assumed [5], [8].

At the same time, connection migration should not be interpreted as implying zero cost or perfectly disruption-free operation. When a peer address changes, QUIC requires path validation on the new path before fully relying on it, and the transport may need to adapt its round-trip-time estimate and congestion state to the characteristics of the new route. As a consequence, HTTP/3 can preserve connection continuity across address changes, but transient performance adjustments may still occur immediately after migration [5].

QUIC also defines its own loss detection and congestion-control mechanisms, rather than inheriting them from operating-system TCP implementations. This means that transport behavior in HTTP/3 is governed by protocol logic specifically designed for QUIC, including its own acknowledgment strategy, loss recovery procedures, and adaptation to path conditions. For the present project, this is particularly relevant because performance under packet loss and network variation is influenced not only by the absence of connection-wide transport blocking, but also by the way QUIC reacts to loss and adjusts its sending behavior over time [5], [12].

Finally, HTTP/3 introduces QPACK for header compression. Because QUIC streams are independent and may be delivered out of order, the HPACK mechanism used in HTTP/2 could not be applied directly without creating blocking dependencies between streams. QPACK was designed to reduce these dependencies, allowing header compression to remain efficient while avoiding unnecessary cross-stream blocking [3], [6], [7], [13].

D. Conceptual Comparison and Expected Performance Impact

Table I summarizes the main architectural differences discussed in the previous subsections. Rather than repeating them, this section focuses on what those differences imply for the experimental scenarios that follow.

The performance gap between the three protocol versions is not constant across conditions. In stable, low-latency environments with small numbers of objects, HTTP/1.1, HTTP/2, and HTTP/3 tend to behave similarly, since the mechanisms that distinguish them simply have less opportunity to matter. The differences become more pronounced as workloads grow more demanding: more concurrent objects, higher latency, or non-trivial packet loss. Under those conditions, transport-layer HoL blocking in HTTP/2 starts to erode the gains of multiplexing, while QUIC’s per-stream loss isolation gives HTTP/3 a structural advantage [6], [7].

This does not mean HTTP/3 always wins. QUIC’s benefits depend heavily on implementation quality, congestion-control tuning, and whether network middleboxes treat UDP traffic fairly. Several studies have found that HTTP/2 outperforms HTTP/3 in low-loss conditions, precisely because the overhead

TABLE I
COMPARISON OF HTTP/1.1, HTTP/2, AND HTTP/3

Feature	HTTP/1.1	HTTP/2	HTTP/3
Transport	TCP	TCP	QUIC/UDP
Session continuity [†]	Per-connection	Per-connection	Via CID
Multiplexing [‡]	No	Yes	Yes
Header compression	None	HPACK	QPACK
HoL blocking	App + Transport	Transport only	Reduced (per stream)
Connection migration	No	No	Yes

[†]HTTP/1.1 and HTTP/2 maintain sessions through persistent TCP sockets; HTTP/3 uses QUIC Connection Identifiers (CIDs), which are independent of the client IP address and port.

[‡]HTTP/1.1 supports pipelining but not true multiplexing: responses must still be delivered in order, and head-of-line blocking remains at the application layer.

of QUIC’s user-space stack is not offset by any transport benefit when the path is clean [6], [7]. The experimental design of this project accounts for this by testing across a range of conditions rather than a single scenario.

Connection migration stands apart from the other differences because it has no counterpart in HTTP/1.1 or HTTP/2. When a client changes its IP address, as happens during a WiFi-to-5G handover, TCP connections break and must be reestablished. QUIC maintains the session through its Connection Identifiers, which are decoupled from the network path [5], [8]. Whether this translates into a measurable reduction in application-level disruption is the central question of the later experimental phases.

The choice of metrics follows directly from this analysis. TTFB captures handshake and connection-establishment overhead, which differs between TCP+TLS and QUIC’s integrated 1-RTT setup [3], [11]. Total transfer time reflects multiplexing efficiency under multi-object workloads, where HoL blocking is most likely to affect HTTP/2 [2], [4], [10]. Goodput under impaired conditions probes QUIC’s loss-recovery and congestion-control behavior relative to TCP [6], [7], [12]. Each metric maps to a specific mechanism; none of them is redundant.

III. PROJECT OBJECTIVES

The main objective of this project is to perform a comparative experimental analysis of HTTP/1.1, HTTP/2, and HTTP/3 under controlled and reproducible network conditions. In accordance with the proposed project theme, the goal is not only to describe the architectural differences among these protocol versions, but also to quantify how those differences translate into measurable effects under distinct classes of scenarios. More specifically, the experimental evaluation will be organized around baseline operation, controlled network impairments, and access-network change during an active session, so that the observed results can be directly related to the main protocol mechanisms of interest.

A first objective is to quantify the behavior of the three protocol versions under controlled conditions of baseline operation, induced latency, packet loss, and bandwidth limitation. The evaluation will focus on metrics such as TTFB,

total transfer time, and, where appropriate, throughput or goodput, since these indicators capture relevant effects related to connection-establishment overhead, multiplexing efficiency, and sensitivity to transport impairments. This part of the project is intended to establish a structured baseline for the comparison and to determine how each protocol reacts as network conditions become progressively less favorable.

A second objective is to assess the practical usefulness of HTTP/3 connection migration in scenarios where the client changes network context during an active communication session. More specifically, the project will study situations in which an ongoing transfer is exposed to an address or access change, ideally corresponding to a transition such as WiFi to 5G. In this context, the purpose is to determine whether HTTP/3 can preserve session continuity with lower disruption, shorter recovery time, and less need for connection reestablishment than HTTP/1.1 and HTTP/2, which rely on TCP and therefore remain tied to the original connection tuple.

A third objective is to identify the network conditions under which HTTP/3 provides the most visible benefits over previous protocol versions. This includes determining whether the advantages of QUIC become more evident mainly in lossy, delayed, or variable environments, or whether the performance gap remains limited when the communication path is stable and transport impairments are mild. In this way, the project aims not only to compare protocol versions, but also to characterize the conditions under which their architectural differences become experimentally significant.

Overall, the project seeks to establish a direct relation between protocol design choices and experimentally observed behavior. Rather than treating HTTP/1.1, HTTP/2, and HTTP/3 as abstract protocol revisions, the work aims to compare them through measurable outcomes obtained under well-defined conditions, thereby clarifying the practical relevance of HTTP/3 and, in particular, the usefulness of connection migration in realistic communication settings.

IV. TOOLS AND COMPONENTS

The experimental platform adopted in this project is based on a virtualized client–server testbed designed to provide controlled conditions, reproducibility, and a clear separation between the endpoints involved in the measurements. More specifically, the setup consists of two virtual machines created in VirtualBox, one acting as the server and the other as the client. Both machines run Ubuntu Server 24.04.4 LTS [21], [22]. In addition to their NAT interfaces, used for package installation and system maintenance, the virtual machines are also connected through a host-only network, which serves as the isolated communication path for the experiments. In the current setup, the server-side host-only address is 192.168.56.101, while the client-side host-only address is 192.168.56.102.

On the server side, the Web service is provided by Caddy. This choice is appropriate for the present project because Caddy offers native support for HTTP/1.1, HTTP/2, and HTTP/3, while also simplifying local HTTPS deployment

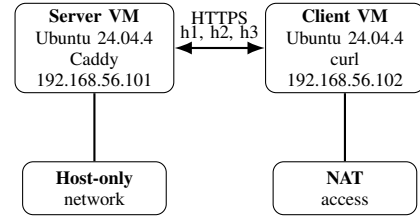


Fig. 1. Virtualized testbed used for preliminary protocol validation.

[15]. In the current implementation, the server is configured to serve the same static HTML resource over HTTPS, using `tls internal` to create a local trust context suitable for preliminary experimentation [16]. Using the same content across the three protocol versions is important because it helps ensure that any differences observed in later measurements can be attributed primarily to transport and protocol behavior rather than to changes in the served application content.

On the client side, the main measurement tool is `curl`. Its use is justified by the fact that it allows explicit protocol selection, controlled request execution, and automatic extraction of timing information such as TTFB and total transfer time [17]. During the preliminary validation stage, it became clear that the default distribution build of `curl` available in the client virtual machine supported HTTP/1.1 and HTTP/2, but not HTTP/3. For this reason, a dedicated `curl` binary was compiled on the client side with support for `nghttp2`, `ngtcp2`, `nghttp3`, and a recent OpenSSL version with QUIC support. This step is itself relevant from the perspective of feasibility, because it confirms that the client-side toolchain can be prepared to execute controlled tests for all three protocol versions within the chosen environment [18], [23]–[26].

Figure 1 summarizes the VM-based testbed adopted for the preliminary validation stage.

For protocol confirmation and lower-layer validation, traffic capture tools such as Wireshark and `tcpdump` are considered part of the experimental toolbox. These tools are important because they make it possible to distinguish TCP-based communication (HTTP/1.1 and HTTP/2) from QUIC/UDP-based communication (HTTP/3), thereby allowing the transport assumptions of the study to be verified directly at packet level [19], [27].

Controlled network impairments will be introduced with `tc/netem` on Linux. This component is essential for the second phase of the project, in which latency, packet loss, and other path variations will be injected in a reproducible way. Since the project aims not only to compare protocol versions under stable conditions but also to study their behavior under more challenging communication scenarios, `tc/netem` provides the mechanism required to create those conditions in a systematic manner [20].

Experiment automation and result processing will be supported by small Python scripts. These scripts are intended to repeat protocol tests, store the outputs generated by `curl`, and prepare the collected data for later analysis and plotting. In this

sense, the overall experimental environment combines virtualization, a multi-protocol Web server, protocol-aware client tooling, packet capture, network emulation, and lightweight automation, which together form a coherent basis for the remaining stages of the project.

From a hardware perspective, the setup requires only a host machine capable of running the two virtual machines reliably during the experiments. For the final migration-oriented phase of the project, the client side will additionally require access to two distinct network paths, ideally WiFi and 5G, so that a path or address change can be introduced during an ongoing transfer. This extension of the setup is important because it will allow the experiments to move beyond static impairment scenarios and directly evaluate the practical usefulness of HTTP/3 connection migration. More specifically, the migration-oriented tests are intended to observe whether an active HTTP/3 session can remain established across the network change, how much temporary disruption is introduced, and how this behavior contrasts with the reconnection that is normally expected from the TCP-based operation of HTTP/1.1 and HTTP/2.

Since this phase depends on the practical availability of two distinct client-side access paths, the final setup may also include an equivalent controlled path-change arrangement if a fully reproducible real WiFi-to-5G transition is not feasible in the available environment. In either case, the experimental objective remains the same: to evaluate whether HTTP/3 can preserve session continuity more effectively than the previous protocol versions when the client network context changes.

V. EVIDENCE OF FEASIBILITY

The preliminary validation reported here had a limited goal: confirm that the selected environment can keep client and server separate, serve content over all three protocol versions, and collect timing measurements reproducibly. It is not a performance study.

A. Experimental Testbed Setup and Connectivity Validation

Two Ubuntu Server 24.04.4 LTS virtual machines were created in VirtualBox, one acting as server and one as client. Each machine has a NAT interface for software installation and a host-only interface for experiments. The server obtained 192.168.56.101 and the client 192.168.56.102 on the host-only network. Successful ping tests in both directions confirmed that the experimental path was operational.

The setup is not localhost. Client and server are distinct machines communicating over an isolated network segment, which is the minimum architecture required for meaningful protocol-level measurements.

B. Server-Side Validation

Caddy was installed on the server and configured to serve a static HTML page over HTTPS at `https://192.168.56.101`. HTTPS is required because HTTP/2 and HTTP/3 negotiation depends on a secure transport context. The `tls internal` option was used to

TABLE II
PRELIMINARY TIMING MEASUREMENTS UNDER LOCAL BASELINE CONDITIONS

Protocol	Runs	Mean TTFB (ms)	Mean total time (ms)
HTTP/1.1	5	8.18	8.20
HTTP/2	5	8.68	9.06
HTTP/3	5	10.17	10.46

generate a local certificate, avoiding the need for external infrastructure while still enabling proper protocol negotiation.

Service status outputs confirmed that HTTP/1.1, HTTP/2, and HTTP/3 were all active on the same server instance. A single Caddy deployment is sufficient to support the full comparative analysis.

C. Protocol Negotiation Validation

The default curl installation on the client supported HTTP/1.1 and HTTP/2 but not HTTP/3. A dedicated binary was compiled with `nghttp2`, `ngtcp2`, `nghttp3`, and OpenSSL with QUIC support [18], [23]–[26].

After compilation, requests with `--http1.1`, `--http2`, and `--http3-only` all returned HTTP 200. The comparative experiment is therefore already executable: the same resource is reachable over all three protocol versions in this environment. The need for a custom build is also worth noting. It is common in experimental work with recent protocol implementations, and the fact that it succeeded here confirms the environment is flexible enough to support it.

D. Preliminary Measurement Capability

Basic timing was collected using `curl -w`, recording `time_starttransfer` and `time_total` for five repetitions per protocol over the host-only path. Results are in Table II.

All three protocols completed in a few milliseconds. HTTP/3 was slightly slower than HTTP/1.1 and HTTP/2, which is expected: over a clean local path with a single small object, QUIC’s user-space stack adds overhead without any transport benefit to offset it. These numbers say nothing about relative performance under realistic conditions. They confirm that the testbed can execute protocol-specific requests and extract timing data consistently.

Baseline values in Table III were collected as separate short reference sets immediately before each `tc` experiment.

E. Preliminary Network-Impairment Validation

To confirm that the experimental path supports controlled impairments, `tc/netem` was applied on the client-side host-only interface (`enp0s8`) to inject 100 ms propagation delay and 1% packet loss [20]. A token bucket filter at 1 mbit/s was used for the capacity-limitation scenario.

All requests completed successfully under each condition, returning HTTP 200. The timing results in Table III show that the injected impairments had a measurable effect on application-level metrics. The delay scenario is the clearest: HTTP/3 completed in roughly 248 ms against 362 ms for

TABLE III
PRELIMINARY TIMINGS UNDER BASELINE AND INDUCED NETWORK
IMPAIRMENTS

Condition	Protocol	TTFB (ms)	Total (ms)
Baseline	HTTP/1.1	9.14	9.18
	HTTP/2	8.03	8.76
	HTTP/3	8.99	9.28
Delay 100 ms	HTTP/1.1	362.36	362.40
	HTTP/2	361.17	361.75
	HTTP/3	248.43	248.49
Loss 1%	HTTP/1.1	40.40	40.47
	HTTP/2	35.40	38.98
	HTTP/3	36.77	38.63
Rate 1 mbit/s	HTTP/1.1	20.43	20.48
	HTTP/2	20.89	23.76
	HTTP/3	31.15	31.26

HTTP/1.1 and HTTP/2, a difference that reflects QUIC’s 1-RTT handshake against the two round-trips required by TCP+TLS. Under loss, differences are smaller and within the noise expected from five-sample sets. Under rate limiting, HTTP/3 was slightly slower, consistent with the earlier observation that QUIC overhead is not offset by transport gains when the object is small and the path is otherwise clean.

These are not performance conclusions. The workload is a single small object, and five samples per condition are not sufficient for statistical claims. What the results do establish is that the testbed responds to controlled impairments in a physically plausible way, which is the prerequisite for the larger experiment campaign that follows.

VI. WORK PLAN

The remaining work is organized in five phases, each building on the previous one. Some overlap between phases is expected, but the ordering reflects the logical dependencies between them.

A. Phase 1: Testbed Stabilization

The immediate next step is to consolidate what has already been validated. This means finalizing the `Caddy` and `curl` configurations, versioning all setup scripts, and documenting the exact build procedure for the HTTP/3-capable `curl` binary. `Wireshark` and `tcpdump` capture will also be integrated at this stage so that protocol negotiation can be verified at packet level during later experiments.

The goal is a reproducible environment that any group member can reset and rerun from scratch. Without that, measurement variance becomes hard to interpret.

B. Phase 2: Baseline Performance Evaluation

With a stable testbed, the first real measurement campaign will establish clean-path baselines for HTTP/1.1, HTTP/2, and HTTP/3. The workload will go beyond the single small object used in the preliminary tests: a page-like scenario with multiple embedded resources will be added, since multiplexing effects in HTTP/2 and HTTP/3 are not visible when only one object is transferred. Metrics will include TTFB, total transfer time, and goodput.

The number of repetitions per condition will be set based on the variance observed in the preliminary results, enough to produce stable means and allow basic statistical comparison between protocols.

C. Phase 3: Controlled Impairment Experiments

Once the baseline is established, `tc/netem` will be used to systematically degrade the experimental path. The planned conditions are added latency (ranging from 20 ms to 200 ms), packet loss (0.1% to 5%), and bandwidth limitation. Each condition will be tested independently and in combination where the interaction is likely to be informative, such as loss under constrained bandwidth.

This is where the architectural differences between the protocols are expected to become experimentally visible. TCP-level HoL blocking in HTTP/2 should worsen under loss; QUIC’s per-stream isolation should limit the damage. Whether the effect is large enough to be statistically meaningful, and under which exact conditions, is what the data will determine.

D. Phase 4: Connection Migration Experiments

This phase addresses the WiFi-to-5G transition scenario specified in the project theme. The planned approach is to run an active HTTP/3 transfer on the client, then change the client’s network interface mid-transfer and observe whether the session survives. For HTTP/1.1 and HTTP/2, the same transition will force a reconnection, which serves as the baseline for comparison.

Two implementations are being considered. The primary approach uses a physical client device with both WiFi and a 5G connection available simultaneously, switching the default route during an active transfer. If that setup cannot be made reproducible, the fallback is a controlled address-change emulated in the VM environment using a second host-only interface and a scripted route switch, which triggers QUIC path validation without requiring physical network infrastructure.

Metrics will focus on session continuity, interruption duration, and recovery time, not just whether the connection survives, but how much it costs in practice.

E. Phase 5: Analysis and Final Delivery

The final phase processes the collected data into comparative figures and tables, and integrates the results into the completed report. The analysis will connect observed behavior to the protocol mechanisms discussed in Section II: handshake design, multiplexing, HoL blocking, and connection migration. Where results are surprising or ambiguous, the discussion will say so rather than fitting them into the expected narrative.

The report will also include a critical assessment of the methodology’s limitations, what the VM-based testbed cannot capture, what the small sample sizes allow and do not allow, and which questions remain open for future work.

REFERENCES

- [1] R. Fielding, Ed., M. Nottingham, Ed., and J. Reschke, Ed., “HTTP/1.1,” RFC 9112, June 2022. doi: 10.17487/RFC9112.

- [2] M. Thomson, Ed. and C. Benfield, Ed., “HTTP/2,” RFC 9113, June 2022. doi: 10.17487/RFC9113.
- [3] M. Bishop, Ed., “HTTP/3,” RFC 9114, June 2022. doi: 10.17487/RFC9114.
- [4] H. de Saxcé, I. Opreescu, and Y. Chen, “Is HTTP/2 really faster than HTTP/1.1?,” in *2015 IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*, 2015, pp. 293–299. doi: 10.1109/INFCOMW.2015.7179400.
- [5] J. Iyengar, Ed. and M. Thomson, Ed., “QUIC: A UDP-Based Multiplexed and Secure Transport,” RFC 9000, May 2021. doi: 10.17487/RFC9000.
- [6] A. Yu and T. A. Benson, “Dissecting Performance of Production QUIC,” in *Proceedings of the Web Conference 2021 (WWW ’21)*, 2021, pp. 1157–1168. doi: 10.1145/3442381.3450103.
- [7] G. Perna, M. Trevisan, D. Giordano, and I. Drago, “A first look at HTTP/3 adoption and performance,” *Computer Communications*, vol. 187, pp. 115–124, 2022. doi: 10.1016/j.comcom.2022.02.005.
- [8] A. Buchet and C. Pelsser, “An Analysis of QUIC Connection Migration in the Wild,” *ACM SIGCOMM Computer Communication Review*, vol. 55, no. 1, pp. 3–9, 2025. doi: 10.1145/3727063.3727066.
- [9] P. Barford and M. Crovella, “A Performance Evaluation of Hyper Text Transfer Protocols,” in *Proceedings of ACM SIGMETRICS ’99*, 1999, pp. 188–197. doi: 10.1145/301464.301560.
- [10] M. Wijnants, R. Marx, P. Quax, and W. Lamotte, “HTTP/2 Prioritization and its Impact on Web Performance,” in *Proceedings of the 2018 World Wide Web Conference (WWW ’18)*, 2018, pp. 1755–1764. doi: 10.1145/3178876.3186181.
- [11] M. Thomson, Ed. and S. Turner, Ed., “Using TLS to Secure QUIC,” RFC 9001, May 2021. doi: 10.17487/RFC9001.
- [12] J. Iyengar, Ed. and I. Swett, Ed., “QUIC Loss Detection and Congestion Control,” RFC 9002, May 2021. doi: 10.17487/RFC9002.
- [13] C. Krasic, M. Bishop, and A. Frindell, Ed., “QPACK: Field Compression for HTTP/3,” RFC 9204, June 2022. doi: 10.17487/RFC9204.
- [14] R. Peon and H. Ruellan, “HPACK: Header Compression for HTTP/2,” RFC 7541, May 2015. doi: 10.17487/RFC7541.
- [15] The Caddy Authors, “Caddy Documentation,” *Caddy Documentation*. Available: <https://caddyserver.com/docs/>.
- [16] The Caddy Authors, “Automatic HTTPS,” *Caddy Documentation*. Available: <https://caddyserver.com/docs/automatic-https>.
- [17] D. Stenberg *et al.*, “curl man page,” *curl Documentation*. Available: <https://curl.se/docs/manpage.html>.
- [18] D. Stenberg *et al.*, “HTTP/3 with curl,” *curl Documentation*. Available: <https://curl.se/docs/http3.html>.
- [19] The Wireshark Foundation, “Wireshark User’s Guide,” *Wireshark Documentation*. Available: https://www.wireshark.org/docs/wsug_html/.
- [20] S. Hemminger, “tc-netem(8) – Linux manual page,” *Linux man-pages*. Available: <https://man7.org/linux/man-pages/man8/tc-netem.8.html>.
- [21] Oracle, “Oracle VirtualBox User Guide,” *VirtualBox Documentation*. Available: <https://www.virtualbox.org/manual/>.
- [22] Canonical, “Ubuntu Server documentation,” *Ubuntu Documentation*. Available: <https://ubuntu.com/server/docs/>.
- [23] T. Tsujikawa *et al.*, “nghttp2 documentation,” *nghttp2 Documentation*. Available: <https://nghttp2.org/documentation/>.
- [24] T. Tsujikawa *et al.*, “ngtcp2 project is an effort to implement IETF QUIC protocol,” *GitHub*. Available: <https://github.com/ngtcp2/ngtcp2>.
- [25] T. Tsujikawa *et al.*, “nghttp3: HTTP/3 library written in C,” *GitHub*. Available: <https://github.com/ngtcp2/nghttp3>.
- [26] The OpenSSL Project Authors, “openssl-quic,” *OpenSSL Documentation*. Available: <https://docs.openssl.org/3.6/man7/openssl-quic/>.
- [27] The tcpdump/libpcap Project, “tcpdump(8) – Linux manual page,” *Linux man-pages*. Available: <https://man7.org/linux/man-pages/man8/tcpdump.8.html>.

Generative artificial intelligence tools were used for conceptual clarification and language support. All final written content and conclusions were independently produced and verified by the authors.